

Práctica del perceptrón múltiple

I. Aproximación de funciones

Las redes neuronales son muy capaces de aproximarse a casi cualquier función en dimensionalidad arbitraria. Para demostrar esto con un ejemplo simple, implementaremos una aproximación de red neuronal para funciones 2D y 3D simples.

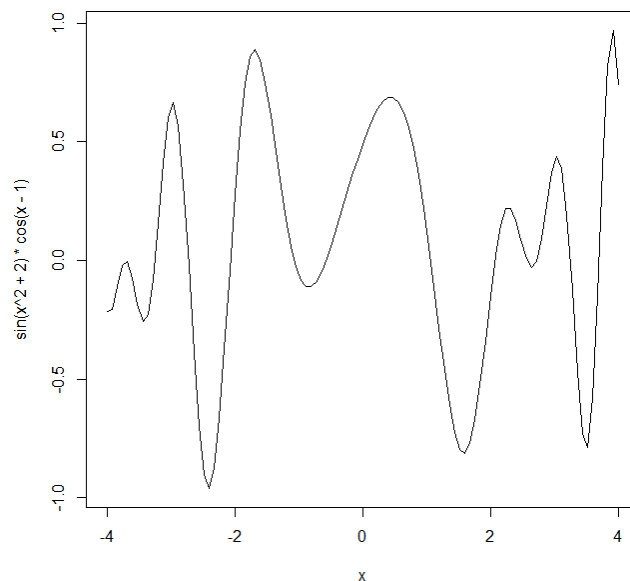
a) Aproximación de una función 2D

La función que consideramos es la siguiente:

$$f(x) = \sin(x^2 + 2) \cdot \cos(x - 1)$$

En primer lugar, debemos visualizar la función directamente. Puedes hacerlo fácilmente a través de la función `curve`.

```
> curve(sin(x^2 + 2) * cos(x-1), -4, 4)
```



Sin embargo, es preferible hacerlo con `ggplot2` por razones de versatilidad:

```
> funct1 <- function(x) sin(x^2 + 2) * cos(x-1)
> p <- ggplot(data.frame(x = c(-4, 4)), aes(x = x)) + stat_function(fun = funct1)
```

Entonces, ¿cómo lograremos la tarea de aproximar esta función? Supongamos que en realidad no se conoce la función de generación de datos $f(x)$, de hecho, solo has tomado muestras de algún proceso que produce esta función.

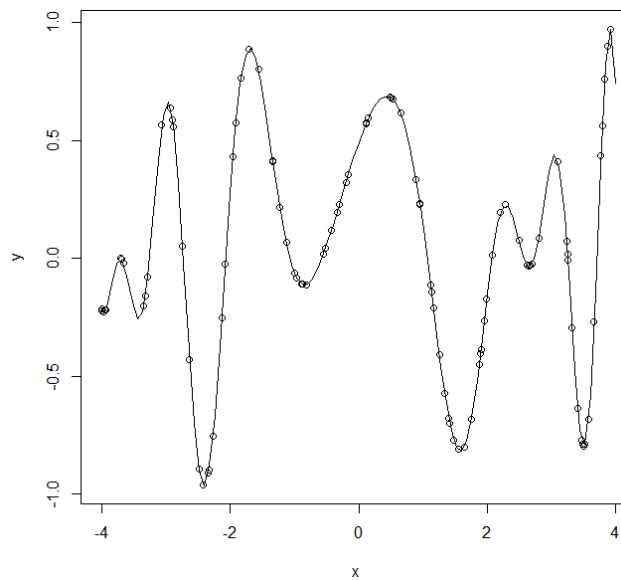
100 muestras de $f(x)$ pueden verse así:

```
> set.seed(42)
> x <- runif(100, min = -4, max = 4)
> y <- funct1(x)
```

En este ejemplo, las muestras de $f(x)$ no contienen ningún ruido. Esta es una configuración muy poco realista, porque los datos del mundo real siempre contendrán el ruido hasta cierto punto. Sin embargo, esto nos permite adaptarnos a una red neuronal sin tener que considerar el sobreajuste (el ajuste excesivo significa que el modelo describe un error aleatorio o ruido en lugar de la relación subyacente). Ten en cuenta que, en realidad, la calibración del modelo al equilibrar el sesgo y la varianza es de vital importancia y no se tiene en cuenta en esta práctica.

Las muestras (x, y) junto con $f(x)$ se ven de la siguiente manera.

```
> plot(x, y)
> curve(sin(x^2 + 2) * cos(x-1), -4, 4, add=TRUE)
```



```
> library(nnet)
```

Primero necesitas instalar el paquete nnet.

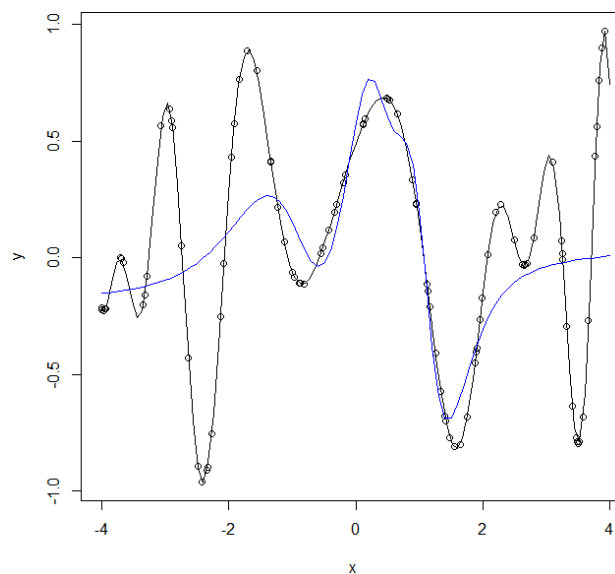
El comando nnet es responsable de entrenar la red neuronal **feedforward**. Detecta automáticamente las dimensiones de las neuronas de entrada y salida (ambas son 1 aquí) y toma un parámetro de tamaño que especifica la cantidad de neuronas ocultas. Para permitir que la red tome en $f(x)$, le damos a la red 20 neuronas ocultas. **El número de iteraciones de aprendizaje se puede cambiar mediante el argumento maxit.**

```
> nn <- nnet(x, y, size=20, maxit=40, linout=TRUE)
# weights: 61
initial value 34.617052
```

```
iter 10 value 18.379923
iter 20 value 16.360417
iter 30 value 15.784461
iter 40 value 15.111218
final value 15.111218
stopped after 40 iterations
```

Cada iteración definida por maxit actualizará los pesos de la red con el fin de minimizar la pérdida de entrenamiento mediante la retropropagación.

```
> grid <- seq(-4, 4, by=0.1)
l> ines(grid, predict(nn, data.frame(x=grid)), col="blue")
```



Como puedes ver, el número de iteraciones controladas por maxit es crucial para el rendimiento de nuestra red 1-20-1. También puedes jugar con el parámetro de tamaño y ver qué pasa. Una vez más, generalmente no es aconsejable minimizar la pérdida del entrenamiento hasta una convergencia como esa, ya que podríamos sobrecargar las funciones ruidosas con bastante facilidad.

Por lo tanto, el uso de conjuntos de datos de prueba y detención temprana es esencial.

Prueba con otras configuraciones para que aproximes correctamente la función!

Realiza tu reporte en base a este ejemplo, debes quizás probar con una variación de los parámetros de la red neuronal.

b) Aproximación de una función 3D

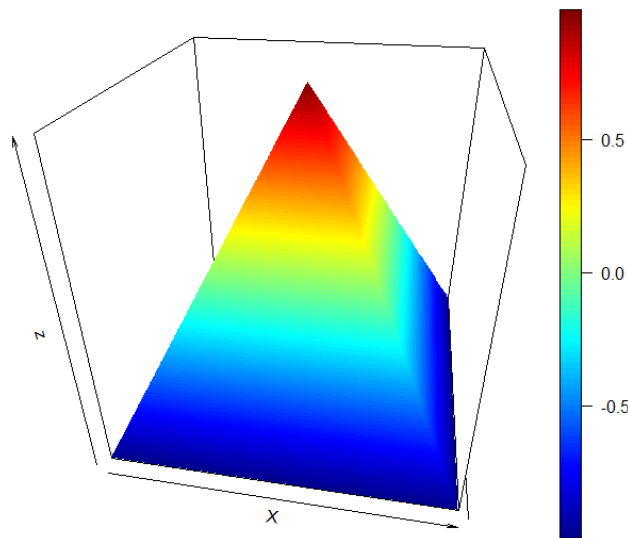
Como segundo ejemplo, vamos a implementar una función con dos valores de entrada (x_1 , x_2) mapeando en espacio 3D. La función que aproximaremos es

$$f(x, y) = 1 - |x + y| - |y - x|$$

Esta función es una pirámide rectangular. De nuevo, tracemos la función con R:

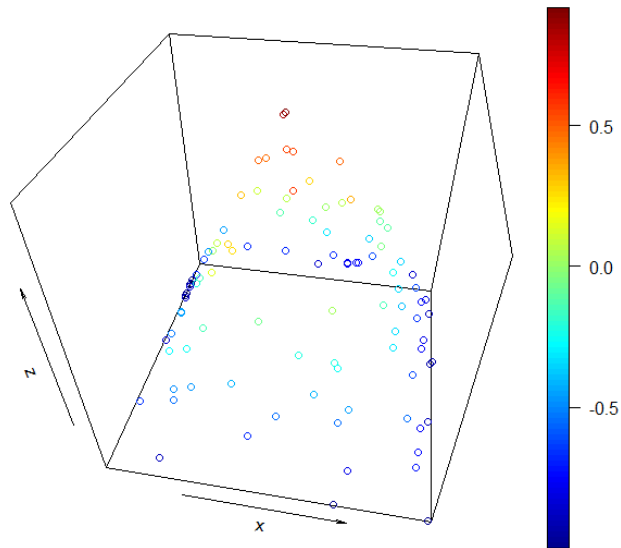
```
> library(nnet)
> library(plot3D) # instalar el paquete plot3D
> set.seed(42)
> x <- seq(-1, 1, by = 0.01)
> y <- seq(-1, 1, by = 0.01)
> grid <- mesh(x, y)
> z <- with(grid, 1-abs(x+y)-abs(y-x))
> persp3D(z=z, x=x, y=y, xlab = "X", ylab = "Y", facets=TRUE, theta=10, phi=30)
```

persp3D (del paquete plot3D) dibuja superficies 3D y toma valores (x , y , z) como entradas. Los parámetros θ y ϕ determinan el punto de vista sobre la trama.



Nuevamente, suponemos que no conocemos esta función y hemos recopilado 100 valores de muestras aleatorias:

```
> funct<-function(x,y) {1-abs(x+y)-abs(y-x)}
> sample_x <- runif(n=100,-1,1)
> sample_y <- runif(n=100,-1,1)
> sample_z <- funct(sample_x,sample_y)
> scatter3D(sample_x,sample_y,sample_z,theta=10, phi=40)
```



Desde este ángulo, no es fácil ver que todos esos puntos se encuentran en la pirámide. Con la ayuda de estos 100 puntos, ahora entraremos en una red neuronal capaz de aproximar $f(x)$:

```
> nn <- nnet(cbind(sample_x, sample_y), sample_z, size=10, maxit = 40, linout = TRUE)
# weights: 41
initial value 55.935494
iter 10 value 15.517893
iter 20 value 3.554909
iter 30 value 1.118576
iter 40 value 0.622092
final value 0.622092
stopped after 40 iterations
```

Para dar cuenta de la creciente complejidad del modelo, le damos 30 neuronas ocultas. Esta vez, los vectores de entrada tienen dos dimensiones - (x, y) y el vector de salida tiene una dimensión - z .

```
> test_vec = expand.grid(x, y) # gives a 40401x2 test vector
> res=predict(nn, test_vec)
```

Para probar la red neuronal en una rejilla, usamos el comando `expand.grid`, que nos da todas las combinaciones posibles de vectores bidimensionales usando los valores 1, 0.99, ..., -0.99, -1 (recuerda, x y y son secuencias de -1 a 1 por 0.01) como un vector de prueba. Dado que nuestros datos solo se encuentran en el mundo 3-D, podemos visualizarlos con bastante facilidad.

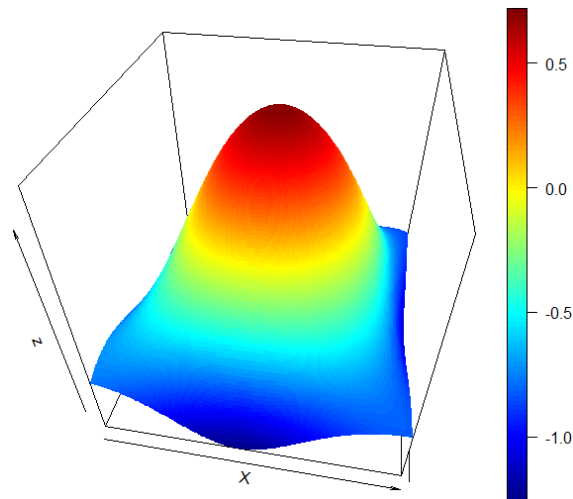
Al alimentar estos valores en la función `nnet.predict`, obtendremos un vector 40401×1 de valores z .

```
> dim(res)<-c(201, 201)
```

```
> persp3D(z = res, x = x, y = y, xlab = "X", ylab = "Y", facets = TRUE, theta = 10, phi = 40)
```

Como `persp3D` toma una rejilla como entrada, transformamos el 40401×1 de valores z en una matriz de 201×201 . De esta manera, podemos trazar una vez más la aproximación de función dada por la red neuronal.

Como en el ejemplo anterior, alteramos el número de iteraciones que ejecuta la red. Aparentemente, alcanzamos un buen ajuste mucho más rápido que en el ejemplo 2-D. Las razones son probablemente una función más fácil de aproximar (incluso si agregas otra dimensión) y un número bastante bajo de puntos de muestra.



Cambiar los parámetros de red, como la cantidad de capas ocultas, probablemente mejore los resultados.

El paquete R `nnet` que se usó aquí no proporciona muchos hiperparámetros o arquitecturas de red avanzadas y es particularmente útil para calcular redes neuronales de vanilla con una sola capa oculta.